Name : Srushti Jayaraman
PRN : 12622427142
Batch : J2

Assignment -2

- **Title**
Implement errors control protocol (error detection and correction Hamming code) : (Language : C / C++ / Python / Java)

- **Objectives**
1. To encode and decode original data bits with the help of parity bits.
2. To demonstrate the use of error control protocols

- **Theory**

1. Types of errors
- Single bit error : only one bit changes
- Burst error : two or more bits are affected

2. Parity Bit
An extra bit added to data to detect errors.
- Even parity : Total is = even
- Odd parity : Total is = odd

3. Hamming code example
Data = 1011
For 4 data bits, 3 parity bits are required
Positions : 1, 2, 3, 4, 5, 6, 7
Bits : P_1 P_2 1 P_4 0 1 1
Using Even Parity
- $P_1 = 0$

Sundaram
FOR EDUCATIONAL USE

- $P_2 = 1$
- $P_4 = 0$

Transmitted = 0110011
Suppose receiver gets 0110111 (error at position 5)
Parity checks give:
$S_4\ S_2\ S_1 = 101 = 5$
So error at position 5
Correct it
0110111 → 0110011
Remove parity & bits → 1011
Hence, Hamming code detects & corrects a single b
error.

- Student Observation
Hamming code provides both error detection & corr

- FAQs

14/8/25

17 What is the difference between flow & error co
(i) Flow control: Controls the rate of data transm
so the receiver is not overlooked.
(ii) Error control: Detects and/or corrects errors
occuring during data transmission.

24 Explain in brief two types of error control
mechanisms
(i) ARQ (Automatic Repeat Request): Detects an e
requests retransmission of the damaged data.
(ii) FEC (Forward Error Correction): Adds redundant
so that receiver can detect and correct err
without transmission.

Sundaram®
FOR EDUCATIONAL USE

```cpp
#include <iostream>
#include <string>
#include<cmath>
using namespace std;

int main() {
    string data;

    cout << "Enter 4 bit binary data: ";
    cin >> data;

    if (data.size() != 4) {
        cout << "Invalid input. Enter exactly 4 bits."
 << endl;
        return 0;
    }

    for (int i = 0; i < 4; i++) {
        if (data[i] != '0' && data[i] != '1') {
            cout <<
"Invalid input. Only 0 and 1 are allowed." << endl;
            return 0;
        }
    }
    int m= data.size();
    int r=0;
    while(pow(2,r) < (m+r+1)){
        r++;
    }
        cout <<"Data Bits:"<< data<<endl;
        cout <<"Number of Parity bits:"<<r<<endl;
     char hamming[8];

    hamming[1] = 'P';
    hamming[2] = 'P';
    hamming[3] = data[3];
    hamming[4] = 'P';
    hamming[5] = data[2];
    hamming[6] = data[1];
    hamming[7] = data[0];

    cout << "\nHamming code positions:" << endl;

```

```
for (int i = 1; i <= 7; i++) {
        cout << "Position " << i << ": " << hamming[i
] << endl;
    }
       for (int p = 1; p <= 4; p = p * 2)
{
    int parity = 0;

    cout << "P" << p << " checks positions: ";

    for (int i = 1; i <= 7; i++)
    {
        int temp = i;
        int bit = p;
        while (bit > 1)
        {
            temp = temp / 2;
            bit = bit / 2;
        }
        if (temp % 2 == 1)
        {
            cout << i << " ";
            if (i != p)
            {
                parity = parity ^ (hamming[i] - '0');
            }
        }
    }

    hamming[p] = parity + '0';
    cout << " -> P" << p << " = " << hamming[p] << endl
;
}

cout << "\nParity Bits:" << endl;
cout << "P1 = " << hamming[1] << endl;
cout << "P2 = " << hamming[2] << endl;
cout << "P3 = " << hamming[4] << endl;

cout << "\nFinal Hamming Code: ";
for (int i = 1; i <= 7; i++)
{
    cout << hamming[i];
}
cout << endl;

}
```

```
Enter 4 bit binary data: 1011
Data Bits:1011
Number of Parity bits:3

Hamming code positions:
Position 1: P
Position 2: P
Position 3: 1
Position 4: P
Position 5: 1
Position 6: 0
Position 7: 1
P1 checks positions: 1 3 5 7  -> P1 = 1
P2 checks positions: 2 3 6 7  -> P2 = 0
P4 checks positions: 4 5 6 7  -> P4 = 0

Parity Bits:
P1 = 1
P2 = 0
P3 = 0

Final Hamming Code: 1010101
```

14/8/26